

VPTQ: 초저비트 벡터 사후 양자화

VPTQ: Extreme Low-bit Vector Post-Training Quantization for Large Language Models

Yifei Liu et al. (2024)

스칼라 하나가 아니라 짧은 weight 패턴을 codebook index로 저장하는 weight-only PTQ

초저비트 weight-only PTQ의 문제

낮은 저장량만으로는 좋은 복원과 빠른 decoding을 보장하지 않는다

제약	왜 어려운가	VPTQ의 대응
2-bit scalar	weight 하나당 네 level만 선택	짧은 vector 패턴을 함께 선택
양자화 순서	앞선 오차가 뒤 weight에 남음	column별 보상 update
배포	복원이 decoding을 지배할 수 있음	decode에서 lookup과 GEMV 결합

VPTQ는 표현력, 오차 전달, 실행 경로를 함께 다룬다

스칼라 양자화와 벡터 양자화

스칼라는 값 하나를, VQ는 길이 v 의 패턴 하나를 codebook에서 고른다

표현 단위	저장하는 것	표현할 수 있는 것
스칼라	각 weight의 level 번호	weight 하나의 근사값
벡터	길이 v vector의 codebook 번호	여러 weight의 공동 패턴
VPTQ	index와 shared centroid	vector 내부의 상관된 모양

VQ의 index는 숫자 하나가 아니라 대표 vector 전체를 가리킨다

두 entry codebook으로 보는 VQ

교육용 예시: $v=2$, codebook은 $C0=(1,1)$, $C1=(-1,-1)$ 두 개다

원래 vector	선택 index	복원 vector	제곱 오차
$v1=(0.9,1.1)$	0	$C0=(1,1)$	0.02
$v2=(-0.8,-1.0)$	1	$C1=(-1,-1)$	0.04
두 vector	0, 1	두 centroid 공유	합계 0.06

두 codebook entry면 index는 vector당 1 bit, scalar당 0.5 index bit다

index bit와 유효 bit 폭은 다르다

유효 bit 폭에는 index뿐 아니라 shared codebook과 선택 경로의 비용이 들어간다

항목	두 entry 교육용 예시	실제 VPTQ에서의 의미
index	1 bit per vector	codebook 크기와 vector 길이에 따라 달라짐
vector 길이	v=2라서 0.5 index bit per scalar	길면 index 비용은 낮아질 수 있음
codebook	두 centroid도 저장	평균 bit 폭에 포함
선택 경로	이 예에는 없음	residual과 outlier 경로가 비용을 더함

2.02-bit는 모든 scalar가 물리적으로 2 bit라는 뜻이 아니다

양자화 오차를 단순 거리보다 민감하게 본다

VPTQ는 weight 변화가 loss에 미치는 영향을 Taylor 근사로 다룬다

- 양자화 뒤 weight 변화는 ΔW 로 생각한다.
- 국소 optimum에서는 1차 gradient 항이 작다고 두고 2차 항을 사용한다.
- Hessian이 큰 방향의 같은 변화는 더 큰 loss 변화로 평가한다.
- full Hessian을 그대로 최적화한다는 주장은 아니다.

2차 정보는 codebook 초기화와 column 보상 설계의 근거다

VPTQ의 핵심 선택: column을 순서대로 고정

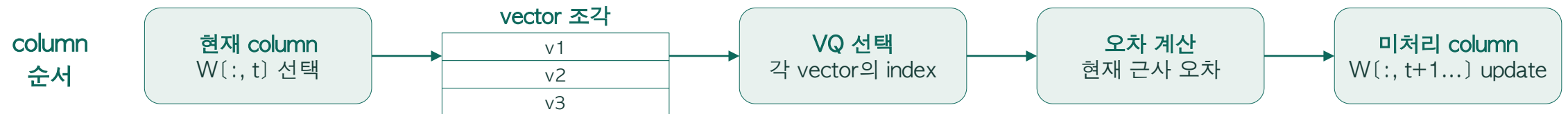
representation은 vector이지만, 처리 순서는 column-by-column이다

처리 방식	현재 단계	뒤 column에 대한 의미
VPTQ	한 column의 vector들을 병렬 양자화	현재 오차를 아직 고정하지 않은 column에 전달
논문 속 GPTVQ 대조	여러 column을 함께 처리	저자는 오차 누적 위험을 문제로 지적
중요한 구분	vector 내부 좌표는 함께 표현	column 처리 순서와 scalar 독립성은 별개

channel-independent는 scalar별 독립 양자화가 아니라 column 처리 순서를 말한다

한 column의 오차를 뒤 column으로 전달한다

교육용 흐름: 현재 column을 고정한 뒤 남은 column만 update한다



이미 고정한 현재 column은 되돌리지 않고, 미래 column만 보상한다

Codebook 초기화에 diagonal Hessian을 쓴다

초기 centroid는 모든 coordinate 오차를 같은 비중으로 보지 않는다

단계	처리	해석
2차 proxy	Hessian을 포함한 근사 오차	민감한 방향을 구분
diagonal 근사	대각 항이 지배적이라고 가정	교차 상호작용을 단순화
weighted k-means	큰 가중치 좌표를 더 중요하게 fit	centroid 초기화에 반영

diagonal Hessian은 편의가 아니라 근사이며 off-diagonal 효과가 사라지는 것은 아니다

VPTQ의 기본 양자화 흐름

가중치 행렬의 column마다 vector 표현과 error compensation을 반복한다

01 Calibration 대표 입력으로 2차 정보 준비



02 Codebook 초기화 diagonal 가중 k-means



03 Column 양자화 길이 v vector를 index화



04 오차 전달 미처리 column update



05 저장 형식 index와 shared codebook

VPTQ는 codebook fitting 하나가 아니라 순서 있는 PTQ 절차다

Residual VQ는 첫 근사의 남은 오차를 다시 code화한다

base centroid와 residual centroid를 더해 더 가까운 vector를 만든다

01	원래 vector	v
↓		
02	Base VQ	첫 centroid C 선택
↓		
03	Residual	$v - C$ 계산
↓		
04	Residual VQ	두 번째 centroid R 선택
↓		
05	복원	$C + R$

Residual VQ는 더 작은 오차와 추가 index·codebook 비용을 교환한다

Outlier와 fine-tuning은 선택 경로다

VPTQ라는 이름 아래에도 구성에 따라 claim boundary가 달라진다

선택 경로	하는 일	발표에서 불일 조건
Outlier elimination	outlier가 있는 tile을 별도 codebook으로 처리	추가 codebook과 bit 비용
Residual VQ	base 근사의 residual을 다시 code화	추가 index와 codebook
Layer-wise fine-tuning	양자화 뒤 layer 단위 조정	순수 PTQ 결과와 구분
End-to-end fine-tuning	전체 모델을 추가 조정	더 강한 결과를 PTQ라고 부르지 않음

표를 읽을 때는 bit 폭뿐 아니라 어떤 선택 경로가 켜졌는지 확인한다

2.02-bit와 2.26-bit의 읽는 법

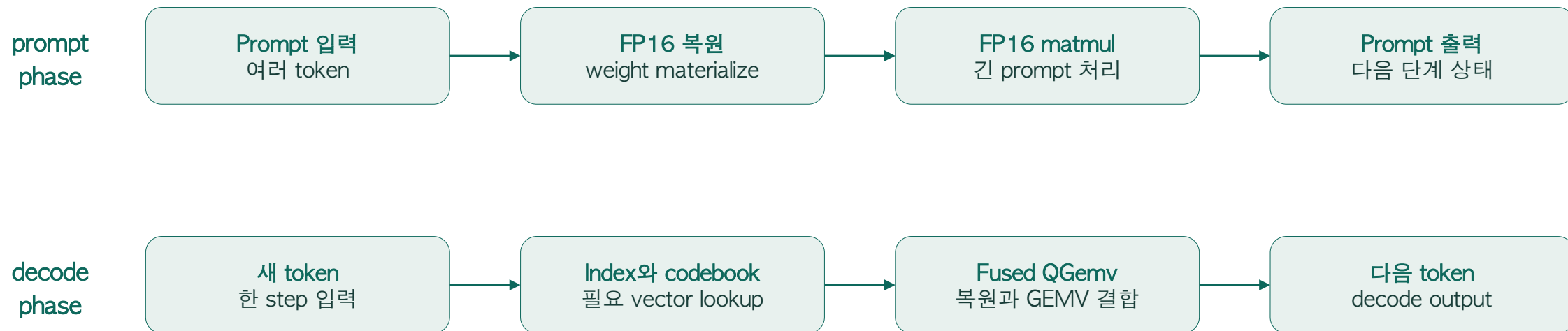
Table 2의 bit label은 codebook·index를 포함해 LLaMA-2 transformer block별로 계산한 format-effective bitwidth다

구성 요소	유효 bit 폭에 포함	잘못된 해석
Base index	vector당 codebook 번호	각 scalar가 동일한 정수 bit
Base codebook	shared centroid 저장	공짜 metadata
Residual과 outlier	선택 시 추가 format 비용	항상 같은 2-bit 형식
집계 단위	LLaMA-2 transformer block별 계산	한 weight의 고정 물리 bit

결과 비교에서는 nominal label보다 effective bit 폭과 memory를 함께 본다

Prompt와 decode는 다른 실행 경로를 쓴다

논문의 low-overhead claim은 decode용 fused QGemv 경로의 조건을 포함한다



decode tok/s는 time-to-first-token이나 전체 요청 latency와 다른 지표다

LLaMA-2 7B의 결과

Table 2의 동일 모델에서 accuracy, memory, decode throughput을 함께 본다

방법	유효 bit	W2 / C4 PPL	tok/s	memory
FP16	16	5.12 / 6.63	38.32	27.22 GB
AQLM	2.29	6.29 / 8.11	19.6	2.40 GB
VPTQ	2.02	6.13 / 8.07	39.9	2.28 GB
VPTQ	2.26	5.95 / 7.87	35.7	2.48 GB

VPTQ 행은 유효 bit 폭을 높이면 PPL이 낮아지는 trade-off를 보인다

LLaMA-2 13B에서도 bit 폭과 속도의 trade-off가 남는다

같은 VPTQ라도 2.02-bit와 2.18-bit의 결과가 서로 다른 operating point다

유효 bit	W2 PPL	C4 PPL	tok/s	memory
2.02	5.32	7.15	26.9	4.03 GB
2.18	5.28	7.04	18.5	4.31 GB

유효 bit 폭을 조금 올려도 정확도와 decode throughput이 같은 방향으로 움직이지 않는다

Column 순서의 ablation

LLaMA-2 13B에서 channel-independent 처리 유무를 비교한 Table 10

설정	v / k	W2 PPL	C4 PPL
Channel-independent: yes	6 / 4096	6.29	8.29
Channel-independent: no	6 / 4096	7.25	9.80

표의 이 조건에서는 column별 error propagation이 더 낮은 PPL과 함께 나타난다

속도와 양자화 비용의 측정 조건

논문의 비교 수치는 특정 phase, GPU, calibration 설정을 가진다

항목	논문의 조건	해석 경계
Calibration	무작위 C4 segment 128개	모든 데이터셋에 충분하다는 보장 없음
Quantization cost	Table 2는 4 x 80GB A100	offline 시간과 decode 시간은 다름
Throughput	decode-phase tok/s	prompt와 전체 request latency가 아님
비교 speedup	AQLM 대비 1.6- 1.8x 보고	7B·13B 표 비교, 일반 속도 법칙 아님

성능 수치는 알고리즘 이름만으로 옮길 수 없고 실행 조건을 함께 가져가야 한다

가까운 방법과 다른 표현 단위

비슷한 저비트 결과라도 무엇을 보존하고 어떤 unit을 code화하는지가 다르다

방법 계열	주 표현 단위	VPTQ와의 구분
RTN과 GPTQ	스칼라 level	VPTQ는 vector centroid index를 사용
GPTVQ	vector codebook	VPTQ는 column별 순서와 error 전달을 강조
AQLM	additive codebook	configuration과 training·runtime 조건이 다름
SqueezeLLM	scalar LUT와 sparse value	VPTQ는 vector codebook과 선택 경로

VPTQ의 차별점은 vector 표현만이 아니라 그 표현을 고정하는 순서다

논문이 남긴 경계와 한계

결과를 이해하려면 approximation, configuration, evaluation 범위를 분리해야 한다

- Diagonal Hessian은 교차 상호작용을 단순화한 근사다.
- Residual, outlier codebook, fine-tuning은 결과의 bit-accuracy boundary를 바꾼다.
- Decode tok/s는 prompt 처리나 end-to-end latency를 뜻하지 않는다.
- 저자는 70B fine-tuning GPU 자원과 일부 LLaMA-3 baseline 부족을 한계로 적었다.

VPTQ는 조건부 empirical 방법이며 모든 모델과 배포 환경의 보편 최적해가 아니다

VPTQ를 한 문장씩 정리하면

저장 형식, 양자화 절차, 실행 경로를 분리하면 논문의 주장이 명확해진다

질문	VPTQ의 답
무엇을 줄이나	weight를 vector index와 shared codebook으로 저장한다
왜 vector인가	낮은 index 비용으로 짧은 weight 패턴을 표현하려 한다
오차는 어떻게 다루나	column을 순서대로 고정하고 미래 column에 보상한다
언제 빠르나	논문 decode phase의 fused QGemm kernel에서
무엇을 확인하나	유효 bit와 옵션, 모델, 지표, 실행 조건

VPTQ의 핵심은 vector codebook, 순차 보상, 조건부 decode 구현의 결합이다